

React.js Complete Notes

1. Introduction to React

- React is a JavaScript library for building user interfaces.
- Developed and maintained by Facebook.
- Component-based architecture.
- Uses Virtual DOM for performance optimization.

2. JSX

- JSX stands for JavaScript XML.
- Allows writing HTML inside JavaScript.
- JSX must return a single parent element.
- Use className instead of class.

3. Components

- Components are reusable pieces of UI.
- Two types: Functional and Class components.
- Functional components are preferred (with Hooks).
- Component names must start with a capital letter.

4. Props

- Props are used to pass data from parent to child.
- Props are read-only.
- Can pass strings, numbers, arrays, objects, and functions.

5. State

- State is used to manage dynamic data.
- useState hook is used in functional components.
- State updates cause re-rendering.
- Never mutate state directly.

6. React Hooks

- useState – Manage state.
- useEffect – Handle side effects.
- useContext – Access context API.
- useRef – Access DOM elements.
- useMemo – Memoize values.
- useCallback – Memoize functions.

7. useEffect

- Runs after component renders.
- Can control execution using dependency array.
- Used for API calls, subscriptions, timers.

8. Forms in React

- Controlled components use state to handle input.
- onChange event updates state.
- Prevent default form submission using event.preventDefault().

9. Conditional Rendering

- Use if-else, ternary operator, or && operator.
- Render UI based on state or props.

10. Lists & Keys

- Use `map()` to render lists.
- Each list item must have a unique key.
- Keys help React identify changed elements.

11. React Router

- Used for navigation in single-page applications.
- `BrowserRouter`, `Routes`, `Route` components.
- `useParams` and `useNavigate` hooks.

12. Performance Optimization

- Use `React.memo` to prevent unnecessary re-renders.
- Use `useMemo` and `useCallback`.
- Lazy loading with `React.lazy` and `Suspense`.

13. Best Practices

- Keep components small and reusable.
- Use proper folder structure.
- Avoid unnecessary state.
- Follow naming conventions.